

2303A51113

B-26

Assignment Number: 3.3(Present assignment number)/24(Total number of assignments)		
Q.No.	Question	Expected Time to complete
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques Lab Objectives • To explore and apply different levels of prompt examples in AI-assisted code generation • To understand how zero-shot, one-shot, and few-shot prompting affect AI output quality • To evaluate the impact of context richness and example quantity on AI performance • To build awareness of prompt strategy effectiveness for different problem types Lab Outcomes (LOs) After completing this lab, students will be able to: • Use zero-shot prompting to instruct AI with minimal context • Use one-shot prompting with a single example to guide AI code generation • Apply few-shot prompting using multiple examples to improve AI responses • Compare AI outputs across different prompting strategies Task 1: Zero-Shot Prompting – Leap Year Check Scenario Zero-shot prompting involves giving instructions without providing examples. Task Description Use zero-shot prompting to instruct an AI tool to generate a Python function that: • Accepts a year as input • Checks whether the given year is a leap year • Returns an appropriate result Note: No input-output examples should be provided in the prompt. Expected Output • AI-generated leap year checking function • Correct logical conditions • Sample input and output • Screenshot of AI-generated response (if required)	Week2 - Wednesday

```

1  >>User> grade > OnDome> >Leap> * Year_checkpy * is_leap_year
2  ***
3  Zero-Shot Prompting: Leap Year Checker
4
5  This module demonstrates zero-shot prompting by providing a function
6  that checks whether a given year is a leap year without prior examples.
7
8  Leap Year Rules:
9  - A year is a leap year if it is divisible by 4
10 - EXCEPT if it is divisible by 100, then it is NOT a leap year
11 - EXCEPT if it is divisible by 400, then it IS a leap year
12 ***
13
14 def is_leap_year(year):
15     """
16     Zero-shot prompted function to check if a year is a leap year.
17
18     A year is considered a leap year if:
19     1. It is divisible by 400, OR
20     2. It is divisible by 4 AND NOT divisible by 100
21
22     Args:
23         year (int): The year to check
24
25     Returns:
26         tuple: (bool, str) - Truth value and descriptive message
27     """
28     if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
29         return True, f"{year} is a leap year (has 366 days)"
30     else:
31         return False, f"{year} is not a leap year (has 365 days)"
32
33
34 def main():
35     """Main function demonstrating the leap year checker with sample inputs."""
36
37     # Sample test cases
38     test_years = [2000, 2004, 2024, 2025, 1900, 2100, 2400, 1996]
39
40     print("\n * 70)
41     print("LEAP YEAR CHECKER - Zero-Shot Prompting Demo")
42     print("\n * 70)
43     print()
44
45     for year in test_years:
46         is_leap, message = is_leap_year(year)
47         status = "✓ LEAP YEAR" if is_leap else "✗ NOT LEAP YEAR"
48         print(f"(Year: {year}): {status}")
49         print(f"    {message}")
50
51     print("\n * 70)
52
53     print("LOGICAL CONDITIONS APPLIED:")
54     print("\n * 70)
55
56     print("1. Divisible by 400? -> Leap Year")
57     print("2. Divisible by 4 AND NOT divisible by 100? -> Leap Year")
58     print("3. Otherwise -> Not a Leap Year")
59     print("\n * 70)
60
61 if __name__ == "__main__":
62     main()

```

Output:

[illegible]**Description:**

1. What it is: Zero-shot prompting leap year checker
2. Input: Accepts a year as input

3. Logic: Divisible by 400 OR (divisible by 4 AND NOT by 100)
4. Output: Returns boolean result with descriptive message
5. Examples: Demonstrates with sample test years

Task 2: One-Shot Prompting – Centimeters to Inches Conversion

Scenario

One-shot prompting guides AI using a single example.

Task Description

Use one-shot prompting by providing one input-output example to generate a Python function that:

- Converts centimeters to inches
- Uses the correct mathematical formula

Example provided in prompt:

Input: 10 cm → Output: 3.94 inches

Expected Output

- Python function with correct conversion logic
- Accurate calculation
- Sample test cases and outputs

```
C:\> Users\pradeep> OneDrive\Pradeep> cmToInch.py ...
1  # One-Shot Prompting: Centimeters to Inches Conversion
2  # Example provided: 10 cm → 3.94 inches
3
4  def cm_to_inches(centimeters):
5      """
6      Converts centimeters to inches using the conversion formula.
7
8      Formula: inches = centimeters / 2.54
9
10     Args:
11         centimeters (float): The value in centimeters to convert
12
13     Returns:
14         float: The equivalent value in inches
15     """
16     inches = centimeters / 2.54
17     return round(inches, 2)
18
19
20 # Test Cases
21 print("Centimeters to Inches Conversion")
22 print("=" * 40)
23
24 # Test case 1: Example provided (10 cm → 3.94 inches)
25 test_value_1 = 10
26 result_1 = cm_to_inches(test_value_1)
27 print(f"Input: {test_value_1} cm → Output: {result_1} inches")
28
29 # Test case 2: 5 cm
30 test_value_2 = 5
31 result_2 = cm_to_inches(test_value_2)
32 print(f"Input: {test_value_2} cm → Output: {result_2} inches")
33
34 # Test case 3: 100 cm (1 meter)
35 test_value_3 = 100
36 result_3 = cm_to_inches(test_value_3)
37 print(f"Input: {test_value_3} cm → Output: {result_3} inches")
38
39 # Test case 4: 2.54 cm (exactly 1 inch)
40 test_value_4 = 2.54
41 result_4 = cm_to_inches(test_value_4)
42 print(f"Input: {test_value_4} cm → Output: {result_4} inches")
43
44 # Test case 5: 30 cm
45 test_value_5 = 30
46 result_5 = cm_to_inches(test_value_5)
47 print(f"Input: {test_value_5} cm → Output: {result_5} inches")
48
49 print("=" * 40)
50 print("Conversion complete!")
51
```

Output:

```

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:/Users/prade/OneDrive/Pradeep/cmToInc.py
Centimeters to Inches Conversion

=====
Centimeters to Inches Conversion
=====
Input: 10 cm + Output: 3.94 inches
Input: 5 cm + Output: 1.97 inches
Input: 10 cm + Output: 3.94 inches
Input: 5 cm + Output: 1.97 inches
Input: 100 cm + Output: 39.37 inches
Input: 2.54 cm + Output: 1.0 inches
Input: 5 cm + Output: 1.97 inches
Input: 100 cm + Output: 39.37 inches
Input: 2.54 cm + Output: 1.0 inches
Input: 30 cm + Output: 11.81 inches
Input: 100 cm + Output: 39.37 inches
Input: 2.54 cm + Output: 1.0 inches
Input: 30 cm + Output: 11.81 inches
=====
Input: 2.54 cm + Output: 1.0 inches
Input: 30 cm + Output: 11.81 inches
=====
Conversion complete!
Input: 30 cm + Output: 11.81 inches
=====
Conversion complete!
=====
Conversion complete!
=====
Conversion complete!

```

Description:

1. The purpose (one-shot prompting demonstration)
2. How it works (single example guidance)
3. What it converts (centimeters to inches)
4. The conversion formula used (1 inch = 2.54 cm)
5. What's included (multiple test cases for verification)

Task 3: Few-Shot Prompting – Name Formatting

Scenario

Few-shot prompting improves accuracy by providing multiple examples.

Task Description

Use few-shot prompting with 2–3 examples to generate a Python function that:

- Accepts a full name as input
- Formats it as “Last, First”

Example formats:

- "John Smith" → "Smith, John"
- "Anita Rao" → "Rao, Anita"

Expected Output

- Well-structured Python function
- Output strictly following example patterns
- Correct handling of names
- Sample inputs and outputs

```

C:\Users\prade>OneDrive\Pradeep> % NameFormatting.py ...
1
2
3 Few-Shot Prompting - Name Formatting
4 This script demonstrates few-shot prompting by providing examples to format names
5 from "First Last" format to "Last, First" format.
6
7
8 # Few-Shot Examples (Prompt)
9
10 examples = [
11     ("input": "John Smith", "output": "Smith, John"),
12     ("input": "Anita Rao", "output": "Rao, Anita"),
13     ("input": "Maria Garcia", "output": "Garcia, Maria"),
14 ]
15
16 def format_name(full_name):
17     """
18     Format a full name from "First Last" to "Last, First" format.
19
20     Args:
21         full_name (str): A full name in "First Last" format
22
23     Returns:
24         str: Formatted name in "Last, First" format
25
26     Examples:
27     >>> format_name("John Smith")
28     'Smith, John'
29     >>> format_name("Anita Rao")
30     'Rao, Anita'
31     """
32     # Strip whitespace and split the name
33     parts = full_name.strip().split()
34
35     # Handle edge cases
36     if len(parts) < 2:
37         return full_name # Return as-is if only one name part
38
39     # Extract first and last name (support for middle names/initials)
40     first_name = parts[0]
41     last_name = parts[-1]
42
43     # Format and return
44     return f"{last_name}, {first_name}"
45
46 def batch_format_names(names):
47     """
48     Format multiple names at once.
49
50     Args:
51         names (list): List of full names
52
53     Returns:
54         list: List of formatted names
55     """
56     return [format_name(name) for name in names]

```

```
C:\Users\prade > OneDrive > Pradeep > formatOfName.py > ...
57
58 if __name__ == "__main__":
59     print("\n" * 50)
60     print("Few-Shot Prompting - Name Formatting")
61     print("\n" * 50)
62
63     # Display few-shot examples
64     print("\nFew-Shot Examples:")
65     print("\n" * 50)
66     for example in examples:
67         print(f"Input: {example['input']}")
68         print(f"Output: {example['output']}")
69         print()
70
71     # Test the function
72     print("Testing the format_name() function:")
73     print("\n" * 50)
74
75     test_names = [
76         "John Smith",
77         "Anita Rao",
78         "Maria Garcia",
79         "Christopher Lee",
80         "Priya Patel",
81     ]
82
83     for name in test_names:
84         formatted = format_name(name)
85         print(f"{name} -> {formatted}")
86
87     print("\n" * 50)
88     print("Batch Processing Example:")
89     print("\n" * 50)
90
91     batch_names = ["Alice Johnson", "Bob Williams", "Carol Davis"]
92     batch_results = batch_format_names(batch_names)
93
94     for original, formatted in zip(batch_names, batch_results):
95         print(f"{original} -> {formatted}")
96
97     print("\n" * 50)
98
```

Output:

```
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe C:\Users\prade\OneDrive\Pradeep\formatOfName.py
Few-Shot Prompting - Name Formatting
=====
Few-Shot Examples:
-----
Input: 'John Smith'
Output: 'Smith, John'

Input: 'Anita Rao'
Output: 'Rao, Anita'

Input: 'Maria Garcia'
Output: 'Garcia, Maria'

Testing the format_name() function:
-----
'John Smith' -> 'Smith, John'
'Anita Rao' -> 'Rao, Anita'
'Maria Garcia' -> 'Garcia, Maria'
'Christopher Lee' -> 'Lee, Christopher'
'Priya Patel' -> 'Patel, Priya'

Batch Processing Example:
-----
Testing the format_name() function:
-----
'John Smith' -> 'Smith, John'
'Anita Rao' -> 'Rao, Anita'
'Maria Garcia' -> 'Garcia, Maria'
'Christopher Lee' -> 'Lee, Christopher'
'Priya Patel' -> 'Patel, Priya'

Batch Processing Example:
-----
'Alice Johnson' -> 'Johnson, Alice'
'Bob Williams' -> 'Williams, Bob'
'Carol Davis' -> 'Davis, Carol'
=====
```

Description:

- The few-shot prompting concept
- How the script converts name formats
- The technical approach used
- Batch processing capability
- The educational value of the examples

Task 4: Comparative Analysis – Zero-Shot vs Few-Shot

Scenario

Different prompt strategies may produce different code quality.

Task Description	Model	Score
1. Task 1: Text Classification	Model A	85%
2. Task 2: Sentiment Analysis	Model B	78%
3. Task 3: Named Entity Recognition	Model C	92%
4. Task 4: Machine Translation	Model D	88%
5. Task 5: Question Answering	Model E	75%
6. Task 6: Image Classification	Model F	90%
7. Task 7: Video Classification	Model G	82%
8. Task 8: Audio Classification	Model H	79%
9. Task 9: Recommendation System	Model I	87%
10. Task 10: Fraud Detection	Model J	91%

- Use zero-shot prompting to generate a function that counts vowels in a string
- Use few-shot prompting for the same problem
- Compare both outputs based on:
 - Accuracy
 - Readability
 - Logical clarity

Expected Output

- Two vowel-counting functions
- Comparison table or short reflection paragraph
- Conclusion on prompt effectiveness

[illegible]

Output:

```
C:\Users\prade>C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python12.exe c:/Users/prade/OneDrive/Pradeep/quant_wexels.py

COMPARATIVE ANALYSIS - ZERO-SHOT VS FEW-SHOT PROMPTING
=====

TEST RESULTS:

Input: 'Hello'
Zero-Shot Result: 2
Few-Shot Result: 2 (breakdown: ('a': 0, 'e': 1, 'i': 0, 'o': 1, 'u': 0))
Results Match: True ✓

Input: 'Programming'
Zero-Shot Result: 3
Few-Shot Result: 3 (breakdown: ('a': 1, 'e': 0, 'i': 1, 'o': 1, 'u': 0))
Results Match: True ✓

Input: 'AIEducation'
Zero-Shot Result: 10
Few-Shot Result: 10 (breakdown: ('a': 2, 'e': 2, 'i': 2, 'o': 2, 'u': 2))
Results Match: True ✓

Input: 'Python is awesome!'
Zero-Shot Result: 6
Few-Shot Result: 6 (breakdown: ('a': 1, 'e': 2, 'i': 1, 'o': 2, 'u': 0))
Results Match: True ✓

Input: 'xyz'
Zero-Shot Result: 0
Few-Shot Result: 0 (breakdown: ('a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0))
Results Match: True ✓
```

Description:

Zero-shot prompting produces straightforward and easily understandable code.

Few-shot prompting results in more concise and optimized solutions.

Both approaches are accurate and logically correct.

Few-shot improves code quality by learning from examples.

Overall, few-shot prompting is more effective for cleaner and professional code, while zero-shot is better for learning basics.

Task 5: Few-Shot Prompting – File Handling

Scenario

File processing requires clear logical understanding.

Task Description

Use few-shot prompting to generate a Python function that:

- Reads a .txt file
- Counts the number of lines in the file
- Returns the line count

Expected Output

- Working Python file-processing function
- Correct line count
- Sample .txt input and output
- AI-assisted logic explanation

```
C:\Users\pradeep> OneDrive\Pradeep > file_handling.py ? ...
def create_and_read_file():
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
How the code works:
1. Opens or creates a file named 'sample.txt' in write mode
2. Writes 6 lines of sample text to the file
3. Closes the file
4. Opens the file again in read mode
5. Counts the total number of lines using a generator expression
6. Prints the result in the required format

Returns:
...int: The total number of lines in the file
...
filename = "sample.txt"

# Sample text to write to the file
sample_lines = [
    "This is the first line of sample text.\n",
    "Python is a great programming language.\n",
    "File handling is an important skill in programming.\n",
    "This program creates and reads a text file.\n",
    "It counts the number of lines automatically.\n",
    "This is the sixth line for extra content.\n"
]

# Create and write to the file
try:
    with open(filename, "w") as file:
        file.writelines(sample_lines)
    print(f"File '{filename}' created successfully.")
except IOError:
    print(f"Error: Unable to create file '{filename}'.")
    return -1

# Read and count lines in the file
try:
    with open(filename, "r") as file:
        line_count = sum(1 for line in file)

    # Print the result in the required format
    print(f"The file '{filename}' has {line_count} lines.")
    return line_count
except FileNotFoundError:
    print(f"Error: File '{filename}' not found.")
    return -1
except IOError:
    print(f"Error: Unable to read file '{filename}'.")
    return -1

# Main program
if __name__ == "__main__":
    print(f"{'-' * 50}")
    print("File Creation and Line Counter Program")
    print(f"{'-' * 50}")
    print()
    create_and_read_file()
    print()
    print(f"{'-' * 50}")
```

Output:

```
Active code page: 65001

C:\Users\pradeep>C:\Users\pradeep\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:/Users/pradeep/OneDrive/Pradeep/file_handling.py
=====
File Creation and Line Counter Program
Active code page: 65001

C:\Users\pradeep>C:\Users\pradeep\AppData\Local\Microsoft\WindowsApps\python3.12.exe c:/Users/pradeep/OneDrive/Pradeep/file_handling.py
=====
File Creation and Line Counter Program
=====

File 'sample.txt' created successfully.
The file 'sample.txt' has 6 lines.

=====
```